Optimized Memory Tagging on AmpereOne® Processors

Shivnandan Kaushik, Mahesh Madhav, Nagi Aboulenein, Jason Bessette, Sandeep Brahmadathan, Benjamin Chaffin, Matthew Erler, Stephan Jourdan, Thomas Maciukenas, Ramya Jayaram Masti, Jon Perry, Massimo Sutera, Scott Tetrick, Bret Toll, David Turley, Carl Worth, Atiq Bajwa
Ampere Computing, Portland, OR and Santa Clara, CA

***Abstract*—Memory-safety escapes continue to form the launching pad for a wide range of security attacks, especially for the substantial base of deployed software that is coded in pointer-based languages such as C/C++. Although compiler and Instruction Set Architecture (ISA) extensions have been introduced to address elements of this issue, the overhead and/or comprehensive applicability have limited broad production deployment. The Memory Tagging Extension (MTE) to the ARM AArch64 Instruction Set Architecture is a valuable tool to address memory-safety escapes; when used in synchronous tag-checking mode, MTE provides deterministic detection and prevention of sequential buffer overflow attacks, and probabilistic detection and prevention of exploits resulting from temporal use-after-free pointer programming bugs.**

The AmpereOne® processor, launched in 2024, is the first datacenter processor to support MTE. Its optimized MTE implementation uniquely incurs no memory capacity overhead for tag storage and provides synchronous tag-checking with single-digit performance impact across a broad range of datacenter class workloads. This paper analyzes the complete hardware-software stack of those workloads, identifying application memory management as the primary remaining source of overhead and highlighting clear opportunities for software optimization. The combination of an efficient hardware foundation and a clear path for software improvement makes the MTE implementation of the AmpereOne® processor highly attractive for deployment in production cloud environments.

I. Introduction

Memory safety continues to be a challenge for pointer based languages such as C/C++, where exploits of memory safety escapes provide a launching pad for a broad range of attacks [1]. The Google Chromium Project's analysis of critical and high severity security issues reported since 2015 attributed over 70% of these to memory safety, roughly evenly split between *spatial* exploits such as buffer overflow and *temporal* bugs such as pointer use-after-free [2]. Similar supporting data published for Microsoft products in a 2019 BlueHat presentation and for Google Android - attributed over 70% [3] and 60% [4] of vulnerabilities, respectively, to memory-safety escapes. While memory-safe languages such as Rust [5, 6] are starting to gain adoption, the transition to these is slower than desired and memory safety remains a challenge for the broad base of C/C++ based software deployed in production environments [7–9].

Software techniques - such as address space layout randomization [10] and stack canaries [11] - for mitigating the impact of attacks exploiting memory-safety escapes have long been known and widely used. Similarly, hardware features such as Intel Control Flow Enforcement Technology (CET) [12] for the x86 architecture and Pointer Authentication (PAC), Branch Target Identification (BTI) [13] for the ARM AArch64 architecture are gaining traction. However, memory-safety violation detection and prevention solutions suitable for large-scale deployment in production environments have remained elusive.

Specifically, software techniques such as Address Sanitizers in compilers (LLVM, GCC) and in the Linux Kernel have been developed for identifying memory-safety escapes during code development and testing [14–16]. Historically, the x86 (Intel Memory Protection Extensions (MPX) [17]) and SPARC (Application Data Integrity (ADI) [18, 19]) architectures have also introduced ISA extensions to detect/prevent memory-safety escapes. However, these techniques have limited usage in production deployments due to their performance and memory footprint impact as well as the need to rewrite some portions of code, incorporating Application Binary Interface (ABI) changes [19]. In late 2025, the ChkTag feature was announced for the x86 architecture as a memory safety ISA extension with details of the architecture extension to become available later [20].

In contrast, the ARM Architecture Memory Tagging Extension (MTE) introduced in ARMv8.5-A in 2018 [21], [22] is established as a promising architecture extension to detect/prevent memory safety escapes. The ISA extension allows associating tag values with pointers and physical memory. Based on comparison of the pointer tag value with the physical memory tag value for reads/writes, MTE enables deterministic detection and prevention of spatial sequential buffer overflow attacks and probabilistic detection and prevention of exploits resulting from temporal use-after-free pointer programming bugs [23]. Memory tag checking can typically be enabled without making any code changes to applications — at most needing to re-link the application with a tagging-enabled memory allocation library. This makes the feature attractive for use in production environments.

The AmpereOne® processor, launched in 2024, is the first datacenter System-on-Chip (SoC) that provides support for ARM MTE [24]. This paper provides details of its MTE implementation, which addresses the key memory capacity and performance overheads that have historically limited the deployment of memory-safety features. Through innovations in tag storage and core microarchitecture, the AmpereOne® processor delivers a hardware foundation designed to minimize the cost of tagging. Benchmarking results demonstrate that on this foundation, datacenter workloads can gain the benefits

979-8-3315-5065-3/26/$31.00 ©2026 IEEE
DOI 10.1109/ISCA66397.2026.00080

of MTE with only a single-digit percentage performance impact on a design that incurs no memory capacity overhead. This result establishes an efficient hardware baseline for what is fundamentally a hardware-software co-design effort. By mitigating the primary hardware costs, this work paves the way for the software ecosystem to further optimize allocators and runtimes, making performant, production-ready memory safety an achievable goal.

This paper is structured as follows. Section II provides an architectural overview of the ARM Memory Tagging Extension. Section III provides an overview of contemporary processor implementations of MTE targeted at handheld devices. Sections IV and V discuss aspects unique to a datacenter SoC MTE implementation and the tradeoffs among microarchitectural options. Section VI provides details of the MTE implementation on the Ampere Computing AmpereOne® processor. Section VII provides details on the performance impact of software use of MTE for a range of datacenter workloads, along with an analysis of the cause of performance deltas. Section VIII includes a discussion of potential future enhancements to the Ampere MTE implementation and implications of future memory technology directions. Finally, Section IX provides concluding thoughts on MTE and the optimized implementation in the AmpereOne® processor.

II. ARM Memory Tagging Extension Overview

The ARM Memory Tagging Extension (MTE) is a memory-safety feature that provides mechanisms for the detection and prevention of the two main classes of memory-safety escapes. Specifically, with appropriate software use of tag values, it provides deterministic detection and prevention of sequential buffer overflow attacks and probabilistic detection and prevention of exploits resulting from use-after-free and similar pointer programming bugs [22]. MTE provides the above capabilities with the following architecture elements:

- Physically addressable memory is partitioned into 16-byte *granules*. Software (typically the `malloc` implementation) assigns a 4-bit *allocation tag* to each granule using new tagging instructions. The architecture does not specify where allocation tags must be stored and leaves that choice to the SoC implementation.
- The pointer used to access the memory contains a 4-bit *address tag* located in the upper unused bits of the virtual address. The address tag is programmed by software (e.g. `malloc`) when a pointer to allocated memory is returned. The ARM architecture provides a top-byte-ignore capability (TBI) [25] to indicate that the highest order byte - in which the address tag is located - not be used by the processor when performing address translation for the virtual address.
- The address tag is checked against the allocation tag before every memory read/write access. The architecture allows for mismatches between the allocation and address tags to be handled either *synchronously* or *asynchronously*.

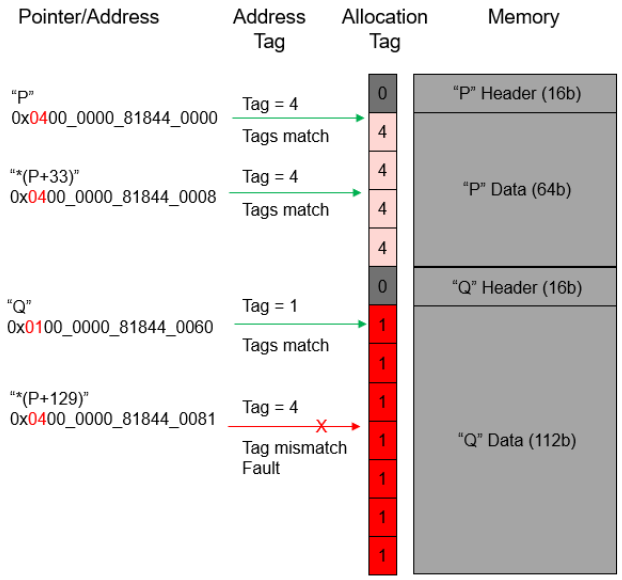

Fig. 1: Buffer Overflow detection and mitigation with Memory Tagging

- The synchronous mode (SYNC) signals an exception on any mismatch and prevents the access from completing, similar to a page or permission fault.
- The asynchronous mode (ASYNC) allows the memory read/write to complete and sets a system register bit to indicate that a tag mismatch has occurred for later examination by software.

With a careful choice of allocation tag values for buffers located sequentially in memory, MTE can detect, and in the synchronous mode, prevent buffer overflow accesses. Given that 4-bits are available for a tag value, memory allocation software reuses tag values resulting in the probabilistic detection and (in synchronous mode) prevention of escapes resulting from pointer programming/temporal use-after-free bugs. Figure 1 illustrates a scenario where use of memory tagging can effectively detect and mitigate a buffer overflow. In the example, there are two separate but sequentially located allocations of memory from a heap - the first returned with a pointer/address P and the second with a pointer/address Q. Memory allocated for pointer P is tagged with allocation tag value of 4 and the tag value 4 is programmed in bits 56:59 of the pointer virtual address as the address tag. Similarly, memory allocated for pointer Q is tagged with an allocation tag value of 1 and the tag value 1 is programmed in bits 56:59 of the pointer virtual address as the address tag. References from pointer P and pointer Q within their allocated memory ranges "pass" since the address tag and the allocation tags match. However, for an access using Pointer P that goes past the end of its allocated memory range into the memory range for allocation Q, the address tag value of 4 does not match the allocation tag value of 1. In SYNC mode, the PE raises a tag mismatch fault and the access does not complete.

Microsoft published an analysis of the effectiveness of the deterministic and probabilistic protection provided by MTE for memory safety escapes reported in their CVEs [26]. The analysis supports the underlying value proposition for MTE, providing:

- Deterministic and durable protection for ~13% of memory-safety escapes reported (in the period covered by the report) associated with "Heap adjacent over-run/over-read" exploits.
- Probabilistic protection for "use-after free" bugs (~26% of vulnerabilities reported in the period covered by the report) with attack probability reduced to 6% if all tag bits are used, and
- Probabilistic protection for "Heap out-of-bounds read or write (non-adjacent)" bugs (~27% of vulnerabilities reported in the period covered by the report) with attack probability reduced to 6% if all tag bits are used.

III. ARM MTE in Mobile Processors

Since the introduction of the ARM MTE architecture extension in 2018, several processors for handheld devices with support for MTE have been brought to market.

The Google Pixel 8 and Pixel 8 Pro [27] phones, launched in October 2023, introduced support for ARM MTE with both synchronous and asynchronous tag-checking modes. MTE support is not enabled by default on the production phone software, but is available as a capability to application developers. MTE in SYNC mode has successfully found use-after-free pointer bugs on these devices [28, 29].

The Apple iPhone 17 and iPhone 17 Pro built on the Apple A19 and A19 Pro processors, respectively, and launched in September 2025, provide comprehensive support for the ARM MTE architecture extension [30]. Leveraging MTE support in the Apple A19 and A19 Pro processors, in conjunction with an extensive software investment in type-aware secure memory allocators and significant micro-architecture investments in the synchronous mode for MTE, the iPhone 17 and iPhone 17 Pro provide a comprehensive, always-on protection against attacks exploiting memory-safety escapes for the kernel and key user-mode processes. The use of the MTE SYNC mode is specifically called out as a requirement for effective protection from memory safety escapes.

Both the Google Pixel 8 and Apple A19 processors have MTE implementations purpose built for handheld devices. Meanwhile, the datacenter platform, and software environment for which the MTE support in the AmpereOne® processor is implemented, has unique differences from a handheld device's platform and software environment. The differences introduce a set of trade-offs, offering opportunities like the assurance of ECC-enabled memory configurations across all datacenter platforms, yet simultaneously posing challenges related to multi-tenancy and datacenter workload sensitivity to any memory bandwidth/latency degradation. Section IV covers details on these considerations when building MTE support in a datacenter targeted SoC - specifically the need for MTE SYNC mode for effective protection from memory safety escapes.

IV. MTE Considerations For datacenter SoCs

The datacenter platform and operating environment create unique challenges while providing opportunities to develop support for MTE in an SoC. This section outlines these along with a discussion of the implications for MTE support.

A. Memory Capacity and Reliability

Financially, DRAM emerges as the most expensive ingredient in a datacenter platform. Meta's internal data reveals that DRAM's share of the total platform cost rose from 15% in their first generation to a notable 35% in their sixth generation platform [31].

DRAM capacity is a key design point for datacenter platforms. Cloud Service Providers (CSPs) provide Platform-as-a-Service (PaaS) capabilities through Virtual Machines (VMs) that are configured for different workload classes with guarantees on the number of cores and amount of memory available to software in each VM profile [32–34]. Cloud service providers' datacenter platforms are designed to have the appropriate ratio of cores to total DRAM populated in the platform to support the optimal configuration of VM profiles from a Total Cost of Ownership (TCO) perspective. Any reduction of memory available to software has a direct impact on the density of provisioned VMs and the resulting cloud providers' TCO.

For the MTE architecture, employing a 4-bit tag per 16 bytes of data, the memory footprint for storing allocation tag values is ~3% of the total software-visible memory ((4 bits / (16 bytes * 8 bits/byte + 4 bits)) = 3.03%). The SoC implementation needs to provide memory for tag storage for the entirety of the memory accessible to software [25] given the potential for all of this memory to be tagged. Consequently, an implementation option that avoids reducing the capacity of software-available memory is highly desirable for MTE's production deployment.

Datacenter environments are characterized by a high volume of platform deployments, each with large memory footprints. Given this scale, robust memory reliability features are paramount to detect and correct memory failures and ensure the necessary up-time for datacenter platforms. Thus, comprehensive ECC support across all DIMM configurations, including SECDED and SymbolECC[1] capabilities within the SoC, is considered a minimum requirement for deployed memory configurations [35].

B. Multi-Tenancy

Data shared by Microsoft indicates that more than 90% of VM profiles supported in Azure datacenters are single vCPU, i.e., a single core is available to software running in the VM [36]. In platforms with a high core count SoC such

[1]SymbolECC is a proprietary Ampere feature, based on the Reed-Solomon algorithm. Its correction properties are comparable to what is known in the industry as Chipkill (IBM), SDDC (Intel), AdvancedECC (HP), STAR ECC (Synopsys), and ExtendedECC (Sun Microsystems).

as the AmpereOne® SoC with up to 192 cores, a very large number of VMs, each with different users/tenants, can be co-located and executing at the same time on the same platform. This multi-tenancy creates unique security challenges. The potential uniqueness of the software image in each VM creates a large surface area for memory-safety escapes. Furthermore, there is a risk that an exploit in one VM can expose other tenants on the same platform if the hypervisor and/or service operating system are exposed. Hence, both detection and immediate prevention of attacks are necessary in production deployments. This implies that the SYNC mode for MTE must be used and the SoC should implement SYNC support in a performant fashion. Production cloud environments already provide strong isolation across VMs, and between VMs and the hypervisor, using a range of primitives including ISA based memory management and virtualization capabilities; so using memory tagging provides an additonal detection and mitigation capability for IOMMU containment for DMA traffic and Confidential Computing technologies such as SEV-SNP [37], TDX [38] and ARM CCA [39]. Also, there is recent growing interest in the use of memory tagging for isolation of un-trusted code from the rest of the system [40].

Multi-tenancy introduces complexities concerning co-located tenants' use of the MTE feature. Unlike the homogeneous environment of a handheld device, where a phone provider has control over MTE usage across the software stack, a cloud environment would enable each tenant to independently configure MTE for their respective software images. From the SoC perspective, enabling MTE for even a single tenant requires the SoC to operate under the assumption that all allocation tags are potentially valid and must be preserved during memory writes. Although tagged memory pages are explicitly identified in page tables in the ARM MTE architecture, commercial operating systems allow for physical memory to be mapped by multiple page tables. Figure 2 illustrates a scenario in which a physical address can be mapped into the address spaces of two different processes, one mapping with tagging enabled and the other without tagging. The first mapping at virtual address A in process 1 could be used to instantiate a valid set of tag values associated with the physical address space. A PE walking the page tables for virtual address Y in process 2 cannot assume that the target physical address does not have valid tag, even though the page table mapping indicates as such. Consequently, the SoC cannot solely rely on page table tagging attributes to determine tag preservation, and must conservatively maintain all tag values. This conservative approach can potentially introduce performance overheads impacting all co-located tenants. Given that various virtual machine profiles offered by cloud service providers include performance guarantees, it is critical for SoC implementations to minimize MTE-induced overhead, typically aiming for single-digit percentage performance impact.

C. Goals for MTE support on the AmpereOne SoC

In summary, the requirements of the datacenter operating environment motivated the following goals for the implementation of MTE on the AmpereOne® SOC:

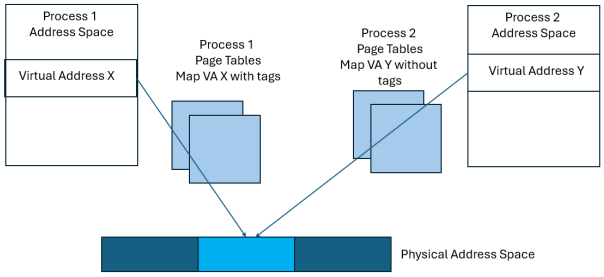

Fig. 2: Necessity for Tag Value preservation by the Processing Element.

- Minimize or eliminate the memory capacity impact associated with MTE allocation tag storage without compromising memory reliability capabilities.
- Support the SYNC mode for tag checking as the primary usage mode, which is essential for production deployments.
- Optimize the micro-architecture implementation to minimize performance impact in SYNC mode.
- Reduce performance degradation for software configurations that do not utilize tag checking, even when MTE is globally enabled on the SoC.

V. Tagging Implementation Options and Tradeoffs

Achieving these goals requires minimizing three categories of overhead:

- *Tag Storage Overhead*: Impact on available memory capacity due to the tag storage mechanism.
- *Tag Fetch Overhead*: Latency introduced by retrieving allocation tags during memory read/write operations for tag checking.
- *Tag Checking Overhead*: Computational cost associated with verifying the congruence between allocation and address tags.

A. Tag Storage Considerations

The organization of MTE allocation tags in memory is a critical design element because it determines the tag storage overhead, and therefore directly affects practicality of MTE deployments. Broadly, there are two ways to organize MTE allocation tags in memory. These are depicted in Figure 3.

Statically allocate sequestered memory for tags: This option involves reserving a dedicated portion of the total physical memory for tag storage at system boot. Given that each 16-byte memory granule requires four bits for its tag, (4 bits / (16 bytes * 8 bits/byte + 4 bits)) = 3.03% of the platform's physical memory must be exclusively reserved for tag data. While methods for partitioning memory at boot are well-established for various specialized usages, their application for MTE tag storage presents significant challenges. As discussed in Section IV, tag storage must be reserved for the entire physical memory present on the platform, due to the infeasibility of predicting which memory regions will be tagged by software

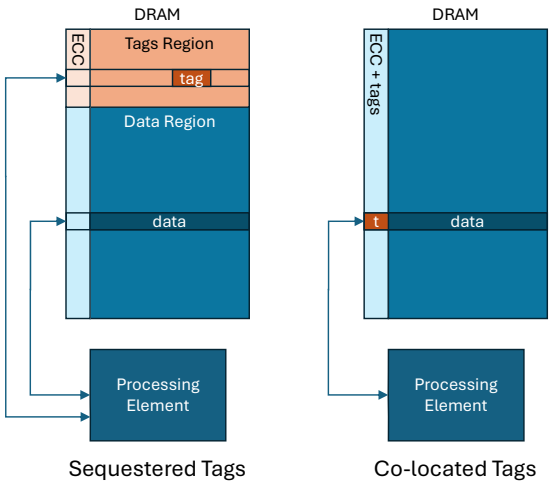

Fig. 3: Comparison of Tag Storage Methods

at runtime. Consequently, sequestering 3% of the total memory directly impacts the density of VMs that a CSP can provision, leading to an increase in TCO. As cloud servers are equipped with terabytes of installed DRAM, this 3% reservation of memory translates into a substantial burden.

Co-locate tag storage with data: This option stores tags alongside the data in dedicated meta-data bits that are not otherwise usable as memory by software. This scheme eliminates the need to carve out software visible physical memory for tag storage up-front, and is favorable to datacenter deployments.

In server platforms populated with memory technology that has ECC support, one option to store allocation tags is to use a subset of the ECC bits. Since these bits are used only by the memory controller unit (MCU) to provide memory reliability features, use of these bits does not impact total memory capacity available to software and eliminates the impact on VM density and TCO degradation to the cloud service provider. AmpereOne® SoC servers use DDR5 registered DIMMs with ECC support providing access of 80 Byte Codewords (64B data + 16B of parity ECC), with a capability to correct a full symbol of 1 Byte. While this allows use of the ECC bits for tag storage, micro-architecture enhancements are needed in the MCU to provide the memory reliability capabilities needed in datacenter platforms using the remainder of the ECC bits available. This support is discussed later in Section VI-B. A similar technique for storing metadata in ECC bits is found in the Sun SPARC ADI implementation for Memory Tagging [18], the RISC-V based Rocket SOC from lowRISC.org [41], architectures for Confidential Computing [42] and GPU memory safety [43].

B. Tag Fetching Considerations

The organization of MTE tags in memory also affects the tag fetch overhead and hence, the run-time performance of applications that use MTE. If tags are not co-located with their data, the CPU read requests to tagged memory will require separate reads to fetch the corresponding allocation tags; these

	Tags Sequestered		Tags Co-Located	
	Coherent Transactions	Memory Transactions	Coherent Transactions	Memory Transactions
Read Data and Tag	2	2	1	1
Write Data and Tag	2	1 + 1 (RMW)	1	1
Read Tag Only	1	1	1	1
Write Tag Only	1	1 (RMW)	1	1 (RMW)
Read Data Only	1	1	1	1
Write Data Only	1	1	1	1 (RMW)

Fig. 4: Transaction Counts per Operation

tag reads consume additional memory bandwidth and also potentially increase memory latency for read operations. CPU reads to untagged memory will not require additional memory reads but may be indirectly affected by the increased traffic due to tagged memory reads. Co-locating tag storage with data allows reading the tags along with the data without generating additional memory traffic and hence, doesn't negatively impact memory bandwidth and latency. Figure 4 shows the relative impact on bandwidth consumption of the MCU and Mesh interconnect on each core cache miss, based on the choice of tag storage method. RMW indicates that the memory subsystem is required to perform a Read-Modify-Write to update the MTE tags. Previous work has demonstrated the performance impact of the increased traffic for memory tagging, in both CPU and GPU architectures [42–44].

C. Tag Checking Considerations

Synchronous tag checking can introduce latency to memory load and store operations, contingent on the specific implementation. This latency arises because tag validation must complete within the CPU pipeline before the memory operation is allowed to commit. This requires pipeline enhancements to perform the check without significantly degrading overall pipeline throughput.

When tags are sequestered from data, both loads and stores can experience additional latency, as tag retrieval requires separate memory transactions. In contrast, if tags are co-located with data, the tag check for load operations incurs virtually no overhead, as tags and data arrive simultaneously. Thus, co-location offers distinct advantages by impacting only store operations with added latency.

A notable disadvantage of co-location, however, is that untagged write-only transactions must be converted into read-modify-write (RMW) operations to preserve existing tags. This conversion can introduce latency and increase bandwidth requirements, particularly if the RMW operation entails loading the cache line into core caches. These overheads may be mitigated by the potentially higher bandwidth of the core to perform RMWs compared to the MCU.

D. Summary

An implementation where MTE allocation tags are co-located with data meets the requirements for MTE usage for production datacenter deployment: no impact on usable

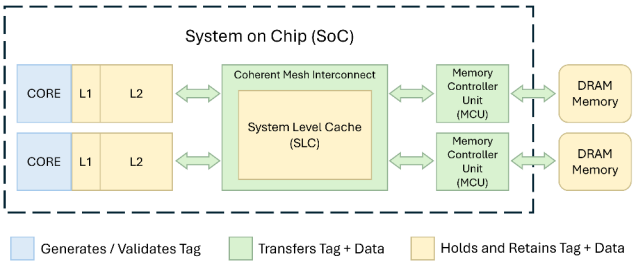

Fig. 5: Ampere MTE System Architectural View

memory capacity, lower hardware complexity, and lower performance overhead. MTE tag storage co-located with data can best be accomplished using ECC bits, but doing so requires enhancements to the memory reliability feature support built on ECC bits to continue providing the high level of memory reliability required for datacenter usages. See Section VI-B for more details.

VI. Ampere's MTE Implementation

The AmpereOne® SoC, a high-performance datacenter server-class processor, offers configurations ranging from 96 to 192 custom Arm v8.6+ ISA-compliant Ampere cores. The AmpereOne® SoC's MTE implementation incorporates enhancements across the Core Processing Element, including the L1 and L2 caches, the coherency mesh/interconnect, and the memory hierarchy, including the system-level cache and the MCU. This section details the MTE support integrated into each of these components, focusing on the design and micro-architectural choices that enable Ampere's efficient and performant implementation of synchronous tag checking.

A. Overview

In Ampere's implementation, tags and data flow together throughout the SoC, the Core Processing Element, and the memory subsystem. This is achieved by ensuring that every subsystem handles the tags and data as a co-located *bundle*. The bundle is first created at the memory subsystem during a memory read when the memory controller reads the tags that are stored in ECC bits inline with the data. The tags are carried over the memory fabric using additional wires and held in caches in dedicated bits which flow through the Core Processing Element pipeline. Figure 5 provides an overview of the SoC components and their roles in generating, retaining, and transporting tags.

B. MTE in the MCU and DRAM

Within the Memory Controller Unit (MCU), ECC bits are utilized to store allocation tags in DRAM. This necessitates the allocation of four ECC bits for every 16-byte DRAM granule. No tag checks are performed within either the MCU or the DRAM itself.

There is no latency impact within the MCU, as the tag bits are transferred along with the data bits, similar to Error-Correcting Code (ECC) bits. However, a notable impact arises concerning the utilization of ECC for error detection and correction. In certain DRAM and ECC configurations, insufficient metadata bits may be available to store MTE tags. To address this, some ECC bits must be re-purposed for tag storage. Ampere accommodates this by offering multiple ECC schemes tailored to various DRAM configurations. Specifically, a novel ECC scheme is employed that provides the requisite reliability protection for datacenter class SoCs while simultaneously making bits available for MTE allocation tags [45]. The selection of the appropriate ECC scheme is determined by both the DIMM configuration and the desired MTE support. AmpereOne® SoC systems support four schemes for ECC:

- **A.** SECDED baseline without MTE support (SECDED-64+8): The code word is 72 bits (64 bits data + 8 bits ECC) with 8 codewords per 64 byte cacheline.
- **B.** SECDED with MTE support (SECDEC-128+4+9): The code word is 128+4+9 bits (128 bits data + 4 bits tag + 9 bits ECC) with 4 codewords per 64B cacheline. This scheme doubles the data granule from (A), allows storage of tag in spare unused ECC bits, retaining full memory reliability feature support without compromise.
- **C.** SymbolECC baseline without MTE support (SymbolECC-64+16): 64 bytes data divided into 8 codewords of 10 nibbles. Reed-Solomon allows one symbol correction per codeword. All the available metadata bits are used for ECC.
- **D.** SymbolECC with MTE support (SymbolECC-64-14+2): Similar to (C) but borrows 2 bits from ECC Parity, resulting in some Correctable errors becoming only Detectable. Compared to (C), this provides 100% of the fault detection, 100% correction of bounded faults, and 99.98% correction of unbounded faults.

For deploying MTE, options **B** and **D** are available to the cloud service provider, and both showcase reliability metrics within the bounds required for datacenter operation.

C. MTE in the Coherent Mesh

Within the mesh, tags are transported using reserved bits that are supported for implementation specific uses. This usage mirrors the use of metadata and ECC bits in the MCU and DRAM with tags being transported along with the data. On the AmpereOne® SoC, this required customization of the mesh to allow for metadata transport along with data. All cache line storage in the mesh is expanded to include tag bits, and this storage is also covered by ECC. No tag checks are performed in the Mesh.

D. MTE in the Core Processing Element (PE)

Within the Core PE, data tags are transported alongside their corresponding data throughout the pipeline via widened datapaths and are stored co-resident with data in the widened caches. Tag validation is performed at the cache-lookup point within the PE pipeline. These checks proceed in parallel with address translation and access-permission checks (e.g., page-fault detection) and therefore do not introduce additional pipeline stages or stalls.

For loads, this design imposes negligible overhead relative to untagged execution because tag validation is integrated into the existing cache-lookup path. For stores, the primary effect is that the memory tag of the target cache line must be retrieved and validated before the store can commit, which constitutes the dominant MTE-related cost in the core. The implementation incorporates mitigations for this extra cost, including early line fetch for tagged stores.

For tagged stores, the cache-line fetch (including its tag) is initiated during address translation, just like for any normal load, which enables out-of-order overlap of the data/tag fetch with other work. As a result, tag check validation for stores can complete far earlier than the commit pipeline stage.

Store-to-load forwarding presents a specific challenge: a younger load may alias an older store whose target line's memory tag has not yet been fetched and validated. To preserve correctness, the allocation tag (address tag) is recorded in the store buffer, and a load is permitted to forward only if its allocation tag matches that of the older store. At the time of forwarding, the outcome of the store's tag check may still be unknown; however, if the allocation tags match, the subsequent tag validation will either succeed for both operations or fault the store (and thus the speculative load), maintaining correctness. Forwarding across a tag-store instruction (which writes the memory tag) is disallowed, as it could violate this invariant property. Thus, many stores experience no additional latency, while others incur a modest delay.

VII. Evaluation of Ampere's MTE Implementation

A. Platform Configuration

The System Under Test (SUT) utilized an Ampere Computing reference platform, designated as Mt. Mitchell [46]. This rack server was equipped with an AmpereOne® M SoC, featuring 192 cores operating at 3.2 GHz. Memory consisted of 512 GB of SK Hynix DDR5 modules, arranged in an 8x64 GB configuration and operating at 5200 MT/s. The configuration supported SymbolECC for RAS capabilities. For I/O, Samsung NVMe Solid State Drives (SSDs) provided storage, and Mellanox ConnectX 100GbE cards managed all network communications. To generate traffic, two 128-core Ampere Altra® Max servers served as client systems. The clients are configured to keep the servers fully active under a P99 service level agreement. Typically, there is a one-to-one correspondence between client and SUT cores to saturate the system for performance measurement.

B. Software Environment and Workloads

The Linux kernel has supported MTE since version 5.1.0 [47], with corresponding support added to GNU Binutils in version 2.45 [48] and to the GNU C Library (glibc) in version 2.33 [49]. Fedora 36 and later distributions ship their kernels with CONFIG_ARM64_MTE enabled by default. For the following experiments, the system under test operated on Fedora 40 with GNU/Linux 6.10.6 and glibc 2.39.

To assess MTE's performance impact, motivated by similar evaluations [50], data was collected from a diverse range of real-world datacenter applications. These are industry-standard benchmarks, widely recognized for datacenter workload characterization and competitive performance evaluation. In all benchmark runs, all available cores on the SoC were fully utilized, to the extent each benchmark allowed.

Memory tagging-specific overheads for tag fetching and checking are identical for workloads running bare-metal or within a VM; no additional VM-exits are introduced by tag setting or checking. We measured the MTE performance in a VM for subset of the workloads early in the AmpereOne program and found that the virtualization overhead (primarily from stage-2 translations) slightly dilutes the MTE performance impact compared to a bare-metal configuration. Hence, the paper focuses on the worse case of the two, bare-metal workloads, to fully expose the performance impact.

A summary of the benchmarks utilized is provided below.

memcached / memtier [51, 52]: memcached is an open source in-memory NoSQL key/value distributed object caching database application. For performance evaluation, a client-server configuration was employed, using memtier_benchmark-v1.3.0 load generators connected over a high-speed network to the server SUT. The SUT executed multiple independent instances of memcached-v1.6.21.

One important aspect of the SUT configuration was the allocation of CPU resources for IRQ handling. The Mellanox ConnectX NIC has 127 IRQs. If these network interrupts were handled by the same cores running memcached, it would lead to performance contention, particularly when running 192 memcached instances. This contention would result in significantly lower throughput and three times higher P99 latency. To mitigate this and adhere to latency SLAs, 20 SUT cores were explicitly dedicated to handling NIC IRQs, which left 172 cores to run memcached instances.

The workload parameters were set to a 64-byte payload size and a 1:10 set-to-get ratio. To generate traffic, two Ampere Altra® Max servers served as client systems, each configured to run 86 concurrent threads, totaling 172 client request threads. This setup aimed for a one-to-one correspondence between client threads and active SUT cores, for optimal system saturation indicative of a well-tuned cloud server.[2]

Redis / memtier [52, 53]: Redis is an open source in-memory NoSQL key/value data store that functions as either a database or an application cache. The experimental setup employed a client-server configuration: multiple client systems, running the memtier_benchmark-v1.3.0 load generation tool, connected over a high-speed network to the System Under Test. The SUT executes multiple, independent, single-threaded instances of Redis-v7.2.0 servers. While Redis workloads typically leverage the jemalloc memory allocation library [54], jemalloc currently does not support memory tagging. Therefore, to evaluate tagging impact, Redis was linked with glibc to use the standard malloc memory allocator.

[2]The other cloud benchmarks' NIC and client setups were similarly tuned to achieve maximum server throughput in the baseline.

vbench (H.264 Video Transcoding Benchmark) [55]: vbench is a benchmark designed for evaluating H.264 video transcoding scenarios in cloud-based video-as-a-service (VaaS) applications. It assesses transcoding performance relevant to typical cloud video workflows, and employs 15 H.264-compressed input files, varying in resolution (480p to 4K) and frame rates (25 to 60 fps). For this analysis, the focus was on two distinct VaaS profiles, Upload and Video On Demand (VOD):

Upload Profile: Measures transcoding speed and output quality for a first-uploaded temporary file. The primary objective is to make the video available for subsequent processing with minimal delay, without degrading the quality of the original input. This profile's reference uses a single-pass encoding with a constant quality target, allowing the encoder to use a high bitrate to maintain quality.

VOD Profile: Simulates scenarios where quality degradation impacts user experience. It strictly mandates that the transcoded quality must not be degraded compared to the reference. This profile's reference is based on an average case, employing a two-pass encoding strategy with a fixed bitrate target, processing in the background for VOD preparation.

nginx / wrk [56, 57]: nginx is an open-source web server commonly employed as a reverse proxy, load balancer, or HTTP cache. To focus on server-side computational overhead, the SUT was configured with Brotli compression and the workload requests initiated server-side Lua scripts [58]. Given this CPU-bound scenario, a one-to-one client-server configuration is employed where a single client running the wrk-v4.1.0 load generator was sufficient to saturate the SUT. The client connected via a high-speed network to the SUT, which ran a single instance of nginx-v1.24 that spawned a worker process per core.

MySQL / sysbench [59, 60]: MySQL is an open source, SQL-compliant, relational database management system (RDBMS). For performance evaluation Sysbench v1.1, a multi-threaded load generation and benchmarking tool, is employed. Sysbench was used to establish a simple database schema, populate database tables, and generate multi-threaded SQL query workloads. These queries were directed to a single instance of the MySQL-v8.0.43 database server, running concurrently with Sysbench on the SUT and communicating via TCP over a local network interface.

PostgreSQL / HammerDB [61, 62]: PostgreSQL is an open source SQL-compliant, object-relational database management system (ORDBMS). For performance evaluation a client system running HammerDB-v4.3, a multi-threaded load generation and benchmarking tool to generate a TPC-C-like workload, was employed. This workload was transmitted over a high-speed network to the SUT, executing a single instance of the PostgreSQL v15.3 database server.

SPEC CPU® 2017 integer base [63]: SPEC CPU was run in refrate mode with 192 copies to saturate the system and estimate SPECrate®2017_int_base. The binaries were built with open-source community gcc-15.1.1 with these base optimization flags: `-O3 -mcpu=ampere1a -flto`.

C. Performance Analysis

Contemporary research [50] has analyzed ARM MTE performance overheads using single-copy SPEC CPU, and a consolidated multi-tenant setup where both the server benchmark and client load generators are hosted on a single machine. In contrast, the measurements presented below are from 192-copy SPEC CPU and fully utilized server platforms with discrete client load generators. While both approaches are valid, this one closely emulates real-world cloud deployments operating at peak utilization, and offers a unique yet complementary perspective on MTE performance.

Three distinct system configurations are used for performance testing. Since MTE can be controlled at two levels, hardware (enabled/disabled in silicon) and software (tag checking enabled/disabled in user mode), this leads to the following three scenarios.

- **A.** MTE Disabled: The feature is disabled in silicon.
- **B.** MTE Enabled, No Tag Checking: MTE is enabled in silicon. The user-mode software does not use the tag checking enabled option for the glibc memory allocation library.
- **C.** MTE Enabled, With Tag Checking: MTE is enabled in silicon. The user-mode software is configured to use the tag checking enabled option for the glibc memory allocation library. Tag checking is enabled by setting the environment variable for SYNC mode: `GLIBC_TUNABLES=glibc.mem.tagging=3`

This framework enables the performance analysis of two critical decisions: one for datacenter operators regarding hardware MTE enablement, and another for applications selecting to utilize MTE's memory safety features. In multi-tenant environments like cloud platforms, enabling MTE in hardware for one application or virtual machine could impose an inherent overhead on other co-located applications, even if they do not utilize MTE's tag checking. This "always-on" hardware overhead (**B** vs. **A**) is a critical consideration for cloud service providers. For an application intending to leverage MTE's safety features, the relevant overhead is the performance degradation when MTE with tag checking is fully active versus when MTE is available but tag checking is disabled (**C** vs. **B**).

To guide these two decisions, both performance ratios across the benchmarks are shown. Figure 6 presents the performance ratios and measurement metrics for each datacenter cloud workload. Figure 7 reports the ratios for each SPEC CPU 2017 benchmark in 192-copy refrate. To account for the inherent run-to-run variability observed in multi-threaded cloud workloads, each benchmark was executed five times. The resulting data were statistically analyzed using `ministat` [64], which compares two small populations to compute performance ratios and their associated error bars using Student's t-test [65]. All statistical comparisons were performed at a 99% confidence level. Results designated as "on par" indicate that no statistically significant performance difference could be discerned between the compared populations.

Performance comparison		Hardware MTE		+ tag enable	
Benchmark	metric	B / A	+/-	C / B	+/-
memcached / memtier	Ops/s	1.076	0.044	on par	
Redis / memtier	Ops/s	on par		on par	
vbench H.264 Upload	Score	0.982	0.002	0.949	0.002
vbench H.264 VOD	Score	0.943	0.008	0.949	0.001
nginx / wrk	Req/s	on par		1.022	0.005
MySQL / sysbench	Req/s	0.963	0.015	0.964	0.014
PostgreSQL / HammerDB	Req/s	on par		on par	

Fig. 6: Performance impact on datacenter workloads. B/A shows the performance ratio when enabling MTE hardware, and C/B is the subsequent ratio after enabling tag checking. Each result is from five runs processed through `ministat`; "on par" indicates no difference proven with 99% confidence.

	Performance comparison		Instructions committed	
	HW MTE	+ tag enable	HW MTE	+ tag enable
Benchmark	B / A	C / B	B / A	C / B
500.perlbench_r	1.000	0.927	1.000	1.003
502.gcc_r	0.942	0.698	1.000	1.073
505.mcf_r	0.974	0.971	1.000	1.001
520.omnetpp_r	0.987	0.913	1.000	1.028
523.xalancbmk_r	0.990	0.855	1.000	1.031
525.x264_r	0.987	0.968	1.000	1.000
531.deepsjeng_r	0.980	1.002	1.000	1.000
541.leela_r	0.977	0.982	1.000	1.006
548.exchange2_r	1.007	0.989	1.000	1.003
557.xz_r	0.995	0.987	1.000	1.000
Geomean	0.984	0.924		

Fig. 7: Performance impact on 192-copy Est. SPEC CPU® 2017 benchmark scores. B/A shows the performance ratio when enabling MTE hardware, and C/B is the subsequent ratio after enabling tag checking. Analysis shows that committed instructions can increase when requesting tag checking.

When looking at the **B** vs. **A** comparisons, enabling MTE in hardware can introduce a small overhead even when applications are not using tagging, due to SoC mechanisms that preserve tag state through the memory hierarchy. This platform-level comparison is relevant for multi-tenant deployments where a cloud provider enables MTE across the fleet. Across the workloads evaluated in Figures 6 and 7, that overhead was in the 1–6% range, while memcached exhibited a 7% improvement (with high variance). In this case, tag-loads effectively become prefetches, since tag-checking is turned off. Since tags and data are co-located, cache lines are brought in to the L1 data cache which happen to be beneficial to the instruction stream being executed. The source of the MTE performance benefit to memcached was confirmed with PMU data, which showed a significant reduction in time spent waiting for L2 misses, without a significant change in other operations which can prefetch lines into the cache, such as hardware prefetches or wrong-path speculation. Practically, these numbers suggest that tenants not asking for tag checking experience only modest impact when the platform enables MTE support for other users.

Next, the comparison of **C** vs. **B**, which is the cost of turning on tag checking for applications. The overall performance impact of tagging is in the mid single-digit percentages with high confidence. Interestingly, in nginx shows a small but measurable speedup. This gain can be attributed to a prefetch-like effect: to perform tag checking, the cache line for a store is fetched earlier in the pipeline (prior to commit), allowing dependent memory operations to benefit from the prefetched line. When execution is limited by store latency and there is headroom in load-side bandwidth, this earlier fetch reduces the critical path and can improve overall throughput. The L2 prefetchers also get trained by tag-loads, which can lead to further benefits.

The primary sources of hardware slowdown stem from first-generation MTE integration in the AmpereOne® microarchitecture and pipelines. First, as also observed by [50], store-to-load forwarding opportunities are reduced for tag-checked stores in the core. Second, the additional tag-read traffic increases pressure on L1D cache read ports, introducing structural hazards in the load/store pipeline. Refinements to address both of these issues have been incorporated into the next generation of Ampere's CPU cores.

Enabling MTE on the SPEC CPU 2017 suite results in a geometric mean performance degradation of 7.6% on SPECrate, as detailed in Figure 7. This figure is heavily influenced by significant regressions in 502.gcc and 523.xalanc, and to a lesser extent, 500.perlbench and 520.omnetpp. Our analysis of these four workloads indicates that the slowdown stems not from the hardware core issues cited above, but rather a combination of the following software factors:

Increased Instructions Committed: The right-most columns in Figure 7 show that the number of retired instructions increases when memory tagging is enabled. This growth is attributable to two principal sources.

First, at user level, `malloc(3)` and related allocator paths in glibc expand to manage tag initialization and maintenance, adding instructions in the hot paths of allocation and deallocation to set allocation tags on newly returned memory and to clear tags on free. This includes the function `libc_mtag_tag_region()` in glibc.

Second, at the kernel interface, glibc disables use of `brk(2)` for its arenas when tagging is requested and instead relies on `mmap(2)`. The `brk(2)` path, which primarily advances the program break to grow the heap (a simple move of a top-of-heap pointer), does not readily provide the page attributes or initialization required for MTE such as setting `PROT_MTE` and preparing tag storage. In contrast, `mmap(2)` allows the allocator to request appropriately protected anonymous mappings, but it incurs higher instruction overhead: selecting a suitable virtual-address range, creating or merging VMAs (Virtual Memory Areas), updating page tables, honoring protection and flags, and triggering first-touch work and tag initialization. The kernel marks these pages as MTE-capable, and tag setup further increases instruction count and latency on first access.

Figure 8 illustrates this shift: with tagging enabled, calls that would have used `brk(2)` are replaced by `mmap(2)`

system call count, 1-copy	brk(2)	mmap(2)
502.gcc Mode B (HW MTE)	32142	94
502.gcc Mode C (+tag enable)	0	28313
520.omnet Mode B (HW MTE)	2147	92
520.omnet Mode C (+tag enable)	0	1937
523.xalanc Mode B (HW MTE)	3609	52
523.xalanc Mode C (+tag enable)	0	3712

Fig. 8: Breakdown of memory allocation system calls on the SPEC CPU benchmarks which show an increase in instructions committed in Mode C.

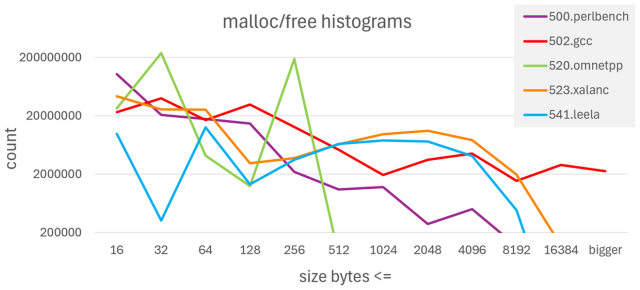

Fig. 9: Distribution of memory allocation sizes and frequencies (log scale) for key benchmarks in SPEC CPU (1-copy). The workloads exhibiting the highest rates of small- to medium-sized allocations directly correspond to those most impacted by MTE overhead. This plot visualizes the allocation-intensive behavior that drives MTE performance degradation.

based arena growth, increasing syscall and kernel work on the allocation path.

Eager Tag Initialization: Our investigation uncovered that 502.gcc and 523.xalanc allocate substantially large virtual address ranges while touching only a fraction of the space. Conventional allocators defer physical-page instantiation until first write; however, when an application requests MTE-tagged regions, these allocators perform eager tag initialization across the entire region before returning. This removes the benefits of lazy population and inflates first-touch costs, disproportionately affecting these workloads. The effect is amplified under homogeneous SPECrate runs, where identical phases execute in lock-step across cores, compounding the adverse behavior.

Transient Small-Object Allocations: The increased instruction execution detailed previously is not uniform; its magnitude depends heavily on an application's memory allocation patterns, specifically the frequency, size, and spatial locality of its allocations. The overhead is most acute in workloads characterized by frequent, small, and transient object allocations that result in a fragmented memory layout. With MTE enabled, each call to malloc incurs a semi-fixed instruction cost to initialize the allocation tag. While this per-call overhead is easily amortized over large, long-lived allocations, it becomes a dominant factor for workloads that perform millions of small, short-lived allocations, directly increasing the user-space instruction count. To validate this hypothesis, we intercepted memory management calls using LD_PRELOAD to profile their frequency and size. Figure 9 plots the counts on a log scale for the five benchmarks exhibiting the largest increase in retired instructions. A strong correlation is evident: workloads with the most significant performance degradation are precisely those with the highest frequency of small-object allocations: 502.gcc, 520.omnetpp, 523.xalanc, followed by 500.perlbench. The other five benchmarks in CPU2017 have allocation counts too low to be visible on the chart's scale.

However, allocation frequency alone is insufficient to explain the full performance impact. For example, 500.perlbench also shows high allocation frequency but experiences a more modest slowdown. This suggests that the spatial pattern of allocations is a critical second factor. We hypothesize that workloads creating a fragmented virtual address space see amplified overheads at both the kernel and hardware levels. A fragmented layout increases the number of distinct VMAs the kernel must manage, making `mmap(2)` calls and page fault handling more expensive. This degrades spatial locality, leading to more TLB misses and costly page table walks.

This hypothesis aligns with the observed behavior. Workloads like 502.gcc, which construct complex, graph-like data structures (e.g., abstract syntax trees), naturally produce a more disjoint and fragmented allocation pattern. This fragmentation is exacerbated by 502.gcc's frequent use of `realloc(3)`. When these reallocations require moving data, their cost is amplified under MTE: the old memory region must be de-tagged and the new region must be tagged, effectively doubling the tag management overhead for a single high-level operation. In contrast, the more linear, stream-like processing in 500.perlbench with zero realloc's results in better memory locality and a less fragmented VMA layout, mitigating some of the kernel- and hardware-level penalties despite also having a high allocation rate of small objects. This finding is consistent with studies of other hardware-based memory safety features; prior research [66] similarly identified the high frequency of heap operations in 502.gcc and 520.omnetpp as the primary performance limiter due to the overhead of extra operations added to the allocator's critical path.

Therefore, the most significant MTE-related slowdowns occur when a high frequency of small transient allocations is combined with a fragmented access pattern within a large memory footprint. This scenario creates a "perfect storm" that stresses many sources of overhead simultaneously: the user-space allocator, kernel memory management, reallocation of memory, and the hardware's address translation mechanisms.

To further validate the hypothesis that frequent, fragmented allocations are a primary source of overhead, we evaluated the performance of these workloads with jemalloc [54], a memory allocator designed to mitigate fragmentation. It accomplishes this through techniques such as size-classed arenas, which are particularly effective for workloads with high rates of small-object allocations. If our hypothesis is correct, the applications impacted by MTE should, in turn, derive benefit from a fragmentation-aware allocator like jemalloc.

Mode B, rate-192	jemalloc uplift
500.perlbench_r	1.044
502.gcc_r	1.030
505.mcf_r	1.055
520.omnetpp_r	1.093
523.xalancbmk_r	1.363
525.x264_r	0.991
531.deepsjeng_r	1.048
541.leela_r	1.006
548.exchange2_r	0.996
557.xz_r	1.008
geomean	1.059

Fig. 10: Performance improvement from linking the `jemalloc` allocator with SPECrate CPU® 2017 benchmarks, running with 192 copies. The significant gains in allocation-intensive workloads provide evidence that MTE performance degradation can be caused by memory management overheads.

Our results confirm this correlation. In Figure 10, we show jemalloc's performance improvement on 192-copy CPU 2017, with a tagging-disabled baseline (since jemalloc does not support tagging). The workloads that exhibited the largest MTE-related regressions (502.gcc, 523.xalanc, and 520.omnetpp) do gain substantial performance when linked and run with jemalloc. This finding provides strong evidence that the observed MTE slowdowns are not an intrinsic cost of tag checking itself, but are tightly coupled to the underlying memory allocation patterns and the efficiency of the memory management subsystem. This study provides motivation for the community to research custom heap allocators with tagging support and smarter policies, which can additionally assist with other types of security issues [67, 68]. Using type-based allocators or size-based allocators also aligns with Apple's direction of using a custom allocator for MTE, xzone malloc [69], where small blocks' tags are reassigned on free but larger blocks' tags are reassigned lazily upon the next allocation [70]. The GNU Tools Cauldron 2025 sessions also acknowledged the issue with MTE and GLIBC malloc of small objects [71].

VIII. Discussion and Future Work

This section outlines future opportunities for improving the performance and scalability of a data-center scale MTE implementation based on the AmpereOne® SoC.

Co-location of tags with data by using a small number of ECC bits for tag storage has significant advantages for a data-center SoC - no memory capacity impact and low performance overhead for a broad range of datacenter class workloads. However, as Sections IV and VII point out, there are impacts to enabling MTE in the platform hardware, even if tag checking is not being used by software. For the AmpereOne® SoC's ECC tag implementation, the small performance delta is due to the need to perform Read-Modify-Write operations when preserving tags for full-cacheline stores that would otherwise be Write-Only. Memory controller designs can consider optimizing this flow further to reduce total overhead. Also, as stated in Section VI, the memory controller implementation in the AmpereOne® SoC for option *D* provides 100% of the fault detection, 100% correction of bounded faults, and 99.98% correction of unbounded faults. As outlined in [45] the "level of 9's" correction of unbounded faults can be increased with additional support in the memory controller implementation. Such improvements can be achieved by carrying error information forward from one codeword to one or more subsequent codewords in the same cacheline. By using this technique, which depends on the extreme unlikelihood of errors occurring in different DRAM devices in the same cacheline, the level of 9's can be improved significantly, to as high as 99.9999%.

Some device classes with no knowledge of tags, such as PCIe devices, have the ability to directly access memory. While such accesses need to preserve allocation tags, doing so for an MTE implementation that co-locates tags with data has a very high degree of complexity for a performant implementation – with an extreme scenario involving read-modify-write transactions for each cache line with the associated significant impact on memory bandwidth. To deliver an optimal system-level solution, the AmpereOne® SoC MTE implementation stipulates that software either not share SW memory intended to be utilized for tagging with devices, or preserve tags around these memory access operations. Looking forward, the ubiquitous presence of System Memory Management Units (SMMUs) in all device-to-memory paths within datacenter SoCs presents an opportunity for architectural advancement. Future SMMU enhancements could incorporate tag awareness, allowing the SMMU to identify and manage device accesses to non-tagged memory regions more optimally. Such an architectural improvement would allow microarchitectural optimizations for these device accesses, thereby eliminating the current software dependence.

The AmpereOne® processor uses bits reserved in the mesh interconnect for metadata to transport tag values alongside the data. The ARM CHI-E extension added explicit support for tag management and transport in the mesh protocol [72]. Future Ampere devices that support the CHI-E mesh protocol will leverage the tag management support for transporting tag values with data using these protocol extensions.

Industry standards such as the Compute Express Link (CXL-3.1) provide profiles for CXL devices with memory, referred to as CXL.mem or CXL Type 3 configurations [73]. Enhancements to the specification for carrying metadata have been defined in the CXL 3.2 specification [74]. Future MTE implementations leverage these extensions to support memory tagging for the CXL.Mem/Type 3 devices. The support requires commercial availability of memory technology devices that support the CXL 3.2 metadata extension along with sufficient storage for tags in the metadata bits.

Similar to the CXL 3.2 extension, it is possible that future DRAM technologies add support for metadata transport. If such standards emerge, future MTE implementations may benefit from leveraging the metadata support for tag transport.

IX. Conclusion

Memory safety remains a major challenge for the large body of software written in pointer-based languages such as C/C++. While compiler and ISA extensions have been proposed to

mitigate these risks, their adoption has been limited by memory capacity and performance overheads. The ARM Memory Tagging Extension (MTE) offers a practical path forward, providing robust, hardware-enforced detection of spatial and temporal memory errors. A key attraction of MTE is its potential for deployment by simply linking applications with a tagging-aware memory allocator. However, as demonstrated in this work, the largest performance overheads stem not from the core MTE mechanism itself, but from the interaction between application allocation patterns and the memory management subsystem.

The AmpereOne® processor addresses the hardware side of this co-design challenge. It integrates system-wide support for ARM MTE while eliminating the memory capacity tax for tag storage and delivering single-digit percent overheads in synchronous mode for a range of datacenter workloads. This is enabled by a co-located data-and-tag design throughout the widened datapaths, caches that store tags alongside data, and core pipeline mechanisms for parallel tag checking – which together provide a highly efficient hardware foundation for memory tagging.

Looking ahead, the path to ubiquitous, low-overhead memory tagging hinges on continued hardware-software co-design. While future hardware will continue to refine the microarchitecture and memory architecture, a significant and immediate opportunity now exists within the software ecosystem. The development of advanced, MTE-aware memory allocators that can mitigate fragmentation and more efficiently manage the lifecycle of tagged transient objects is the essential next step. By building on the performant hardware platform presented here, the software community can unlock the full potential of MTE, making broadly deployed, low-overhead memory safety a reality for production data-center environments.

Acknowledgement

This work would not have been possible without the contributions of many of our colleagues, including the designers and verification engineers that implemented and validated MTE. We would like to specifically thank our performance analysis team: Olivier Singla, Jiangning Liu, Feng Xue, James Burke, Sean Daniels, and Steve Winiecki for conducting extensive performance measurements, and Syed Fakhri for his support of the performance monitoring infrastructure. We are also grateful to Richard Shannon, Mark Charney, Shahid Khan, and Shravan Narayan for their valuable guidance and reviews.

References

[1] L. Szekeres, M. Payer, T. Wei, and D. Song, "SoK: Eternal War in Memory," in *2013 IEEE Symposium on Security and Privacy*, 2013, pp. 48–62. DOI: 10.1109/SP.2013.13.

[2] Google. "The Chromium Projects: Chromium Security: Memory Safety." [Online]. Available: https://www.chromium.org/Home/chromium-security/memory-safety/.

[3] M. Miller. "Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape." [Online]. Available: https://github.com/Microsoft/MSRC-Security-Research/blob/master/presentations/2019_02_BlueHatIL/2019_02%20-%20BlueHatIL%20-%20Trends%2C%20challenge%2C%20and%20shifts%20in%20software%20vulnerability%20mitigation.pdf.

[4] Google. "Memory Safety: Memory Unsafety." [Online]. Available: https://source.android.com/docs/security/test/memory-safety.

[5] S. Klabnik, C. Nichols, and C. Krycho. "The Rust Programming Language." [Online]. Available: https://doc.rust-lang.org/stable/book/index.html.

[6] "The Rust Language Reference." [Online]. Available: https://github.com/rust-lang/reference.

[7] T. Claburn. "In Rust We Trust: Microsoft Azure CTO shuns C and C++." [Online]. Available: https://www.theregister.com/2022/09/20/rust_microsoft_c/.

[8] S. Vaughan-Nichols. "Linus Torvalds talks AI, Rust adoption, and why the Linux kernel is the only thing that matters." [Online]. Available: https://www.zdnet.com/article/linus-torvalds-talks-ai-rust-adoption-and-why-the-linux-kernel-is-the-only-thing-that-matters.

[9] R. Al, C. Carruth, J. Engel, and A. Qin. "Safer with Google: Advancing Memory Safety." [Online]. Available: https://security.googleblog.com/2024/10/safer-with-google-advancing-memory.html.

[10] "Address Space Layout Randomization." [Online]. Available: https://pax.grsecurity.net/docs/aslr.txt.

[11] C. Cowan, C. Pu, D. Maier, H. Hintony, J. Walpole, P. Bakke, S. Beattie, A. Grier, P. Wagle, and Q. Zhang, "Stackguard: Automatic adaptive detection and prevention of buffer-overflow attacks," in *Proceedings of the 7th Conference on USENIX Security Symposium - Volume 7*, ser. SSYM'98, USENIX Association, 1998. [Online]. Available: https://api.semanticscholar.org/CorpusID:2358856.

[12] V. Shanbhogue, D. Gupta, and R. Sahita, "Security Analysis of Processor Instruction Set Architecture for Enforcing Control-Flow Integrity," in *HASP'19: Proceedings of the 8th International Workshop on Hardware and Architectural Support for Security and Privacy*, 2019. DOI: 10.1145/3337167.3337175.

[13] ARM. "Better Security at the Flick of a (Compiler) Switch: Enabling Pointer Authentication and Branch Target Identification." [Online]. Available: https://newsroom.arm.com/blog/pac-bti.

[14] "Clang 22.0.0.git documentation: Address Sanitizer." [Online]. Available: https://clang.llvm.org/docs/AddressSanitizer.html.

[15] "Program Instrumentation Options." [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Instrumentation-Options.html.

[16] "Kernel Address Sanitizer (KASAN)." [Online]. Available: https://www.kernel.org/doc/html/latest/dev-tools/kasan.html.

[17] Intel. "Support for Intel® Memory Protection Extensions (Intel® MPX) Technology." [Online]. Available: https://www.intel.com/content/www/us/en/support/articles/000059823/processors.html.

[18] "SPARC Architecture: Application Data Integrity." [Online]. Available: https://www.kernel.org/doc/html/v6.13-rc7/arch/sparc/adi.html.

[19] K. Serebryany, E. Stepanov, A. Shylapnikov, V. Tsyrklevich, and D. Vyukov, *Memory Tagging and How it Improves C/C++ Memory Safety*, 2018. arXiv: 1802.09517 [cs.CR].

[20] A. Evangelista. "ChkTag: x86 Memory Safety." [Online]. Available: https://community.intel.com/t5/Blogs/Tech-Innovation/open-intel/ChkTag-x86-Memory-Safety/post/1721490.

[21] M. Gretto-Dann. "Arm A-Profile Architecture Developments 2018: Armv8.5-A." [Online]. Available: https://developer.arm.com/community/arm-community-blogs/b/architectures-and-processors-blog/posts/arm-a-profile-architecture-2018-developments-armv85a.

[22] ARM. "Armv8.5-A Memory Tagging Extension." [Online]. Available: https://developer.arm.com/documentation/102925/0100.

[23] H. Liljestrand, C. Chinea, R. Denis-Courmont, J.-E. Ekberg, and N. Asokan, *Color My World: Deterministic Tagging for Memory Safety*, 2022. arXiv: 2204.03781 [cs.CR].

[24] Ampere. "AmpereOne-M® Product Brief." [Online]. Available: https://amperecomputing.com/briefs/ampereone-m-product-brief.

[25] ARM. "ARM Architecture Manual for A-profile Architecture, Section D8.8 "Address Tagging"." [Online]. Available: https://developer.arm.com/documentation/ddi0487/latest.

[26] J. Bialek, K. Johnson, M. Miller, and T. Chen. "Security Analysis of Memory Tagging." [Online]. Available: https://github.com/microsoft/MSRC-Security-Research/blob/master/papers/2020/Security%20analysis%20of%20memory%20tagging.pdf.

[27] B. Rakowski. "Meet Pixel 8 and Pixel 8 Pro, our newest phones." [Online]. Available: https://blog.google/products/pixel/google-pixel-8-pro/.

[28] M. Brand. "First handset with MTE on the market." [Online]. Available: https://googleprojectzero.blogspot.com/2023/11/first-handset-with-mte-on-market.html.

[29] R. Lopez Mendez. "Memory Tagging Extension on Google Pixel 8." [Online]. Available: https://learn.arm.com/learning-paths/mobile-graphics-and-gaming/mte_on_pixel8/.

[30] Apple Security Engineering and Architecture. "Memory Integrity Enforcement: A complete vision for memory safety in Apple devices." [Online]. Available: https://security.apple.com/blog/memory-integrity-enforcement/.

[31] J. Weiner, N. Agarwal, D. Schatzberg, L. Yang, H. Wang, B. Sanouillet, B. Sharma, T. Heo, and M. Jain, "TMO: Transparent Memory Offloading in Datacenters," in *ASPLOS'22: Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022. DOI: 10.1145/3503222.3507731.

[32] Microsoft. "Sizes for virtual machines in Azure." [Online]. Available: https://learn.microsoft.com/en-us/azure/virtual-machines/sizes/overview.

[33] Google. "Google Cloud: Machine families resource and comparison guide." [Online]. Available: https://cloud.google.com/compute/docs/machine-resource.

[34] Oracle. "Oracle Cloud Infrastructure Documentation: Compute Shapes." [Online]. Available: https://docs.oracle.com/en-us/iaas/Content/Compute/References/computeshapes.htm.

[35] J. Lee, M. J. Kim, W.-S. Kim, and Y. S. Kim, "Review of Memory RAS for Data Centers," *IEEE Access*, vol. 11, pp. 124 782–124 796, 2023. DOI: 10.1109/ACCESS.2023.3329984.

[36] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, "Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms," in *SOSP'17: Proceedings of the 26th Symposium on Operating Systems Principles*, 2017. DOI: 10.1145/3132747.3132772.

[37] AMD. "AMD SEV_SNP: Strengthening VM Isolation with Integrity Protection and More." [Online]. Available: https://docs.amd.com/v/u/en-US/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more.

[38] Intel. "Intel Trusted Domain Extensions (Intel TDX)." [Online]. Available: https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/overview.html.

[39] ARM. "Confidential Computing Architecture." [Online]. Available: https://www.arm.com/architecture/security-features/arm-confidential-compute-architecture.

[40] J. Seo, J. You, Y. Cho, Y. Cho, D. Kwon, and Y. Paek, "SFItag: Efficient Software Fault Isolation with Memory Tagging for ARM Kernel Extensions," in *Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security*, ser. ASIA CCS '23, Melbourne, VIC, Australia: ACM, 2023, pp. 469–480, ISBN: 9798400700989. DOI: 10.1145/3579856.3590341.

[41] W. Song. "lowRISC tagged memory tutorial." [Online]. Available: https://lowrisc.org/docs/tagged-memory-v0-1/.

[42] R. Avanzi, I. Mihalcea, D. Schall, H. Montaner, and A. Sandberg, *Hardware-Supported Cryptographic Protection of Random Access Memory*, Cryptology ePrint Archive, Paper 2022/1472, 2022. [Online]. Available: https://eprint.iacr.org/2022/1472.

[43] M. B. Sullivan, M. T. I. Ziad, A. Jaleel, and S. W. Keckler, "Implicit Memory Tagging: No-Overhead Memory Safety Using Alias-Free Tagged ECC," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA '23, Orlando, FL, USA: ACM, 2023. DOI: 10.1145/3579371.3589102.

[44] L. Lamster, M. Unterguggenberger, D. Schrammel, and S. Mangard, "Voodoo: Memory Tagging, Authenticated Encryption, and Error Correction through MAGIC," in *33rd USENIX Security Symposium (USENIX Security 24)*, Philadelphia, PA: USENIX Association, Aug. 2024, ISBN: 978-1-939133-44-1. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/lamster.

[45] M. Sutera, N. Aboulenein, and S. Brahmadathan, "Integrated error correction code (ECC) and parity protection in memory control circuits for increased memory utilization," U.S. Patent No. 12,204,410, 2022. [Online]. Available: https://patents.google.com/patent/US12204410B2/en.

[46] Ampere Computing. "AmpereOne® Family 2U Mt. Mitchell Reference Platform Brief." [Online]. Available: https://amperecomputing.com/customer-connect/products/mt-mitchell.

[47] C. M. Vincenzo Frascino. "Memory Tagging Extension (MTE) in AArch64 Linux." [Online]. Available: https://docs.kernel.org/arch/arm64/memory-tagging-extension.html.

[48] "Memory Tagging Extension (MTE) in AArch64 Linux." [Online]. Available: https://gcc.gnu.org/wiki/MTE.

[49] "Memory Related Tunables." [Online]. Available: https://sourceware.org/glibc/manual/2.33/html_node/Memory-Related-Tunables.html.

[50] T. Noh, Y. Wang, T. Garfinkel, M. Madhav, D. Moghimi, M. Erez, and S. Narayan, "ARM MTE Performance in Practice," in *35th USENIX Security Symposium (USENIX Security '26)*, USENIX Association, 2026. arXiv: 2601.11786 [cs.CR].

[51] "memcached: distributed memory object caching system." [Online]. Available: https://memcached.org.

[52] "NoSQL Redis and Memcache traffic generation and benchmarking tool." [Online]. Available: https://github.com/RedisLabs/memtier_benchmark.

[53] "Redis: The Real Time Data Platform." [Online]. Available: https://redis.io.

[54] J. Evans, "A Scalable Concurrent malloc(3) Implementation for FreeBSD," in *Proceedings of the BSDCan Conference*, Ottawa, Canada, Jan. 2006. [Online]. Available: https://papers.freebsd.org/2006/bsdcan/evans-jemalloc.files/evans-jemalloc-paper.pdf.

[55] A. Lottarini, A. Ramirez, J. Coburn, M. A. Kim, P. Ranganathan, D. Stodolsky, and M. Wachsler, "vbench: Benchmarking Video Transcoding in the Cloud," ser. ASPLOS '18, Williamsburg, VA, USA: ACM, 2018, pp. 797–809. DOI: 10.1145/3173162.3173207.

[56] "nginx project page." [Online]. Available: https://nginx.org.

[57] W. Glozer. "wrk - a HTTP benchmarking tool." [Online]. Available: https://github.com/wg/wrk.

[58] V. Krasnov. "ARM Takes Wing: Qualcomm vs. Intel CPU comparison." [Online]. Available: https://blog.cloudflare.com/arm-takes-wing/#nginx.

[59] "MySQL." [Online]. Available: https://www.mysql.com.

[60] A. Kopytov. "sysbench: Scriptable database and system performance benchmark." [Online]. Available: https://github.com/akopytov/sysbench.

[61] "PostgreSQL: The World's Most Advanced Open Source Relational Database." [Online]. Available: https://www.postgresql.org.

[62] "HammerDB: The industry standard open-source database benchmark." [Online]. Available: https://www.hammerdb.com.

[63] J. Bucek, K.-D. Lange, and J. v. Kistowski, "SPEC CPU2017: Next-Generation Compute Benchmark," in *Companion of the 2018 ACM/SPEC International Conference on Performance Engineering*, ser. ICPE '18, Berlin, Germany: ACM, 2018, pp. 41–42, ISBN: 9781450356299. DOI: 10.1145/3185768.3185771.

[64] P.-H. Kamp. "ministat – statistics utility." [Online]. Available: https://man.freebsd.org/cgi/man.cgi?query=ministat.

[65] W. S. Gosset. "Student's T-Test." [Online]. Available: https://en.wikipedia.org/wiki/Student%27s_t-test.

[66] Y. Kim, J. Lee, and H. Kim, "Hardware-based Always-On Heap Memory Safety," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 1153–1166. DOI: 10.1109/MICRO50266.2020.00095.

[67] DARKNAVY.org. "Strengthening the Shield: MTE in Heap Allocators." [Online]. Available: https://www.darknavy.org/blog/strengthening_the_shield_mte_in_memory_allocators/.

[68] llvm.org, *Scudo hardened allocator*, https://llvm.org/docs/ScudoHardenedAllocator.html.

[69] Apple. "xzone malloc." [Online]. Available: https://github.com/apple-oss-distributions/libmalloc/blob/libmalloc-792.41.1/doc/xzone_malloc.md#MTE.

[70] M. Shavrick, A. Rebert, L. Dionne, and K. Varlamov, "Practical Security in Production," *acmqueue*, vol. 23, no. 5, 2026, ISSN: 1542-7730. [Online]. Available: https://spawn-queue.acm.org/doi/10.1145/3773097.

[71] Community. "GNU Tools Cauldron 2025 (malloc for MTE is slow for small objects)." [Online]. Available: https://youtu.be/s_rsoe65a64?si=syzAsdr05h98Bz_9&t=1141.

[72] ARM. "AMBA CHI Architecture Specification." [Online]. Available: https://developer.arm.com/documentation/ihi0050/h.

[73] "Compute Express Link: About CXL." [Online]. Available: https://computeexpresslink.org/about-cxl/.

[74] "CXL Consortium Announces Compute Express Link 3.2 Specification Release." [Online]. Available: https://computeexpresslink.org/wp-content/uploads/2024/12/CXL_3.2-Spec-Announcement_FINAL-1.pdf.